

INPUT PICTURE:



0

OUTPUT:

```
1] 14s !pip install ultralytics -q
    _____ 42.1/42.1 kB 1.6 MB/s eta 0:00:00
    _____ 1.4/1.4 MB 19.0 MB/s eta 0:00:00
    _____ 53.2/53.2 kB 2.7 MB/s eta 0:00:00

2] 12s ▶ from google.colab import files
    uploaded = files.upload()
    image_path = list(uploaded.keys())[0]

    ... Choose Files images (1).jpeg
    images (1).jpeg(image/jpeg) - 32846 bytes, last modified: 7/9/2026 - 100% done
    Saving images (1).jpeg to images (1).jpeg
```

+ Code + Text

```
yolo_model = YOLO("yolo26n.pt")
results = yolo_model(image_path)
result = results[0]

detected_objects = []
for box in result.boxes:
    cls_id = int(box.cls[0])
    label = yolo_model.names[cls_id]
    conf = float(box.conf[0])
    detected_objects.append((label, conf))

print("Detected objects:")
for label, conf in detected_objects:
    print(f" {label}: {conf:.3f}")

result.save(filename="detected.jpg")

... Creating new Ultralytics Settings v0.0.6 file
View Ultralytics Settings with 'yolo settings' or at '/root/.config/ultralytics/settings.json'
Update settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo26n.pt to 'yolo26n.pt': 100% 5.3MB 45.5MB/s 0.1s

image 1/1 /content/images (1).jpeg: 448x640 6 persons, 6 chairs, 1 potted plant, 564.7ms
Speed: 26.6ms preprocess, 564.7ms inference, 8.1ms postprocess per image at shape (1, 3, 448, 640)
Detected objects:
  person: 0.883
  person: 0.806
  person: 0.782
  chair: 0.762
  chair: 0.746
  person: 0.706
```

```
person: 0.559
chair: 0.370
chair: 0.365
chair: 0.349
potted plant: 0.310
'detected.jpg'

import torch, urllib.request
from torchvision import transforms, models
from PIL import Image

urllib.request.urlretrieve(
    'http://places2.csail.mit.edu/models_places365/resnet18_places365.pth.tar',
    'resnet18_places365.pth.tar')
urllib.request.urlretrieve(
    'https://raw.githubusercontent.com/csailvision/places365/master/categories_places365.txt',
    'categories_places365.txt')

classes = [line.strip().split(' ')[0][3:] for line in open('categories_places365.txt')]

scene_model = models.resnet18(num_classes=365)
checkpoint = torch.load('resnet18_places365.pth.tar', map_location='cpu')
state_dict = {k.replace('module.', ''): v for k, v in checkpoint['state_dict'].items()}
scene_model.load_state_dict(state_dict)
scene_model.eval()

preprocess = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.428], [0.229, 0.224, 0.225])])
```

```

transforms.Resize((224, 224)),
transforms.ToTensor(),
transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

img = Image.open(image_path).convert('RGB')
input_tensor = preprocess(img).unsqueeze(0)

with torch.no_grad():
    probs = torch.nn.functional.softmax(scene_model(input_tensor), dim=1)[0]

top5_prob, top5_idx = probs.topk(5)
scene_predictions = [(classes[idx], float(prob)) for prob, idx in zip(top5_prob, top5_idx)]

print("Scene/background predictions:")
for scene, prob in scene_predictions:
    print(f"  {scene}: {prob:.3f}")

```

```

.. Scene/background predictions:
   classroom: 0.939
   lecture_room: 0.047
   computer_room: 0.004
   art_school: 0.003
   library/indoor: 0.003

```

```

category_map = {
    "hospital": {"objects": ["person", "bed"],
                     "scenes": ["hospital_room", "operating_room", "medical_center", "hospital"]},
    "classroom": {"objects": ["person", "chair", "laptop", "book", "backpack"],
                  "scenes": ["classroom", "lecture_room", "school"]},
    "roadside": {"objects": ["car", "truck", "bus", "motorcycle", "traffic light", "stop sign"],
                 "scenes": ["highway", "street", "gas_station", "parking_lot"]},
}

```

```

def compute_category_scores(detected_objects, scene_predictions, category_map,
                           obj_weight=0.5, scene_weight=0.5):
    scores = {cat: 0.0 for cat in category_map}
    for label, conf in detected_objects:
        for cat, refs in category_map.items():
            if label in refs["objects"]:
                scores[cat] += obj_weight * conf
    for scene, prob in scene_predictions:
        for cat, refs in category_map.items():
            if any(s in scene for s in refs["scenes"]):
                scores[cat] += scene_weight * prob
    return scores

```

```

scores = compute_category_scores(detected_objects, scene_predictions, category_map)
final_category = max(scores, key=scores.get) if max(scores.values()) > 0 else "uncategorized"

print("Combined category scores:", scores)
print("Final assigned category:", final_category)

```

```

Combined category scores: {'hospital': 2.1942517161369324, 'classroom': 4.273686100845225, 'roadside': 0.0}
Final assigned category: classroom

```

Terminal

Final assigned category: **CLASS ROOM**